Source: docs\CUSTOMER_ORDER_HISTORY_DESIGN.md | Export: 11 November 2026

Customer My Orders

Local implementation of the approved desktop/mobile My Orders references. Scope is `/account/orders`; Profile, Confirmation, Tracking, Checkout and Admin keep their established screens. No production changes were performed.

Source and privacy

- `CustomerOrderHistory.tsx` uses the existing authenticated `customerAction` and `/api/catalog`, action `orders:list`, with opt-in `view: 'cards'`.
- `server/orders.ts` derives ownership from the verified token UID. Every query includes `orders.uid == authenticated UID`. Cards reject client identity fields; legacy callers retain their existing contract.
- The card projection in `historyModel.ts` returns reference, date, status, totals, recorded collection/refund amounts, courier/estimated date, at most two saved item previews, product-line count and total units. It does not expose profile/address/email, actors, internal notes, request reasons or financial reference metadata. Missing legacy item fields remain explicitly unavailable.
- Historical names, variants, photographs and line amounts come from saved order snapshots. There is no catalog lookup per card and archived products do not hide orders.
- Reads do not place orders, reconcile payments, update stock or award/redeem points.

Filtering, pagination and return navigation

Filter	Existing domain statuses
All Orders	All owned orders
In Progress	Placed, Confirmed, Processing, Shipped
Delivered	Delivered
Cancelled	Cancelled, Rejected; the actual individual status remains visible

Filters are applied in Firestore before pagination. Each page returns at most 25 cards, using 26-document lookahead and `createdAt DESC, __name__ DESC` ordering. The client appends pages without duplicates and retains loaded cards on next-page failure. A filter change resets pagination and aborts/ignores the previous request. Counts displayed beneath cards describe loaded orders only; there are no fabricated whole-history filter counts.

Filter URLs use `/account/orders?status=progress|delivered|cancelled`. Actual order links target `/order-tracking/{id}` and support normal modified clicks/new tabs. Back to Profile targets `/account`. Empty-history Explore Sarees targets `/shop`.

The list keeps one UID-scoped, five-minute in-memory return snapshot, keyed by its browser history entry. Returning from Details restores loaded pages and scroll; browser Back/Forward is supported. A fresh/direct entry fetches normally. Full reload discards in-memory pages. Sign-out/account switching clears cached customer data even while Details is mounted. History state contains only a navigation path/key, not orders or tokens. Private list data is never stored in local/session storage. Details' existing Back button uses a strictly validated list return path; direct Details visits fall back to `/account/orders`.

Presentation

- Desktop: horizontal cards with paired portrait previews on the left and amount/payment, shipment summary and full-width action on the right.
- Mobile: one compact portrait preview, accurate additional-line indicator, bounded reference/date/status header, total/payment and View action. At 320px, long references truncate visually while the complete reference remains accessible and copyable.
- Portrait containers are 78×104px desktop, 76×101px mobile; images use `object-fit: contain` and cannot expand cards to their intrinsic width. Missing/failed images have a neutral fallback.
- Lora throughout, maroon `#780D24`, warm ivory `#FBF7F0`, restrained gold. Styles use new history classes. The approved shared logo/header/footer and Home · Shop · Wishlist · Bag · Account navigation remain intact, with Account active.
- Skeletons, true empty history, no filter matches, initial failure, refresh, next-page failure and Retry are distinct. Technical diagnostics remain out of customer UI.
- Delivered and collected are independent. COD due remains due after delivery until collection is recorded. Closed orders show historical Order Value and no requested delivery payment. Collection/refund amounts appear only when recorded; absence is not converted to a fabricated paid/refunded amount.

Necessary reference differences: preserved the established shared storefront shell and actual order references; omitted sample membership/collection counts, Silk Mark/insurance/atelier claims, fictional transit percentages and unimplemented invoice, proof-of-delivery, review, reorder and payment actions. The list contains only a shipment summary; full tracking and assistance stay in the existing Details screen.

Local validation

- Typecheck (`npm run lint`), production build (`npm run build`), 146 unit/query/security regression checks (`npm test`). Build retains the pre-existing large-chunk advisory.
- `customer-history-emulator.test.ts`: exported API handler, signed-out rejection, token ownership, cross-account cursor/detail isolation, bounded snapshots, all filter groups across pages, newest-first ties, genuine empty response, invalid requests and unchanged order revisions.
- Existing `orders-list-emulator.test.ts` and `order-details-emulator.test.ts`: dual administrator authorization, administrative list/query/pagination and historical detail/collection/refund regressions.
- `customer-history-browser.test.py`: real emulator-backed list; snapshot previews and counts; delivered/due versus collected; closed-order totals; pagination and next-page failure/retry; loading/empty/filter-empty; technical error redaction; stale-filter response cleanup; Details links without celebration; list/filter/scroll restoration and Back/Forward; sign-out/account switch; widths 320, 390, 768 and 1440; bounded images and no horizontal overflow.
- Screenshots and equivalent-width Stitch comparisons: `.cache/customer-history/orders-{320,390,768,1440}.png` and `comparison-orders-{390,1440}.png`. Visual captures show three representative real emulator cards; other loaded rows are hidden only during screenshot capture and restored immediately for functional tests. Fixture photography is local; no live customer/order records or media are used.

Release dependencies

Release the frontend and updated catalog server together. No new packages/environment variables, rules, index definitions, migration or production records are required by these source changes.

The manifest already contains these COLLECTION-scope `orders` indexes:

1. `uid ASCENDING, createdAt DESCENDING` for All Orders.
2. `uid ASCENDING, status ASCENDING, createdAt DESCENDING` for all grouped filters (`status in [...]`).

Both inherit `__name__ DESCENDING`, matching explicit query ordering. This task did not inspect or publish live indexes. Production must have these exact definitions READY; emulator/query-manifest tests do not establish live readiness. If either is missing, publish only the missing definition through the owner's normal reviewed release process. No rules publication is required.

Hosted acceptance remains: after the owner releases frontend/server, sign in with an existing customer, verify all status groups, pagination, Details and return navigation. Nothing was committed, pushed, deployed, published or changed in production.